Module 1: Introduction to Backend Development

What is Backend Development?

- **Backend Development** refers to the server-side development of a web application. It is responsible for handling business logic, processing user requests, interacting with databases, managing authentication, and sending responses to the frontend.
- The backend operates behind the scenes and ensures that data is processed, stored, and delivered correctly to users.

Key Responsibilities of Backend Development:

- Handling client requests and server responses
- Managing databases and data storage
- Implementing business logic
- User authentication and authorization
- Building and maintaining APIs
- Ensuring security and performance

How Web Applications Work:

- A web application is a software application that runs on a web server and is accessed through a web browser. It follows a client-server architecture, where the frontend (client) communicates with the backend (server) to process requests and retrieve data.

Client

The **Client** is the user-facing part of the application, typically a web browser or mobile app.

Responsibilities:

- Displays the user interface.
- Collects user input.
- Sends requests to the server.
- Displays responses received from the server.

Examples: Chrome, Firefox, Mobile Apps.

Server

The **Server** is responsible for processing requests and executing business logic.

Responsibilities:

- Receives client requests.
- Processes application logic.
- Handles authentication and authorization.
- Communicates with the database.
- Sends responses back to the client.

Examples: Django, Flask, FastAPI, Node.js.

Database

A **Database** is used to store and manage application data.

Responsibilities:

- Stores user and application data.
- Retrieves requested information.
- Updates, inserts, and deletes records.
- Ensures data consistency and integrity.

Examples: PostgreSQL, MySQL, MongoDB.

API (Application Programming Interface)

An **API** is a communication bridge between the client and the server.

Responsibilities:

- Receives requests from the client.
- Transfers data between client and server.
- Returns responses in formats such as JSON.
- Enables integration with other applications and services.

Examples:

- Client requests user details.
- API sends the request to the server.
- Server fetches data from the database.
- API returns the data to the client.

Frontend vs Backend:

Frontend	**Backend**
User-facing part of the application	Server-side part of the application.
Focuses on UI and user experience.	Focuses on business logic and data processing.
Runs in the user's browser.	Runs on the server.
Handles user interactions.	Handles requests, authentication, and database operations.
Communicates with the backend through APIs.	Communicates with databases and external services.
Technologies: HTML, CSS, JavaScript, React, Angular	Technologies: Python, Django, Flask, Node.js, Java, Spring Boot.

Example

When a user logs in:

Frontend:

- Displays the login form.

- Collects username and password.
- Sends login request to the backend.

Backend:

- Validates credentials.
- Checks user data in the database.
- Returns success or failure response.

What is Python?

Python is a high-level, interpreted, object-oriented, and general-purpose programming language known for its simple syntax, readability, and versatility.

It is widely used for web development, data science, automation, artificial intelligence, machine learning, and software development.

Key Features

- Easy to learn and read
- Interpreted language
- Object-Oriented Programming (OOP) support
- Cross-platform compatibility
- Large standard library
- Open-source and community-driven

Applications of Python

- Web Development (Django, Flask, FastAPI)
- Data Analysis and Visualization
- Machine Learning and AI
- Automation and Scripting
- Desktop Applications
- API Development ***Python Program Example***

```
name = input("Enter your name: ")

print("Hello,", name)

print("Welcome to Python Programming!")
```

Output

```
Enter your name: Srinadh

Hello, Srinadh

Welcome to Python Programming!
```

Why Python is Used in Enterprise Applications:

Python is widely used in enterprise applications because it enables organizations to build scalable, maintainable, and efficient software solutions quickly.

Reasons for Using Python in Enterprise Applications

- **High Productivity:** Simple and readable syntax reduces development time.
- **Rapid Development:** Allows faster application development and deployment.
- **Scalability:** Supports the development of small to large-scale enterprise systems.
- **Extensive Libraries:** Provides a vast ecosystem of libraries and frameworks.
- **Cross-Platform Compatibility:** Runs on Windows, Linux, and macOS.
- **Strong Framework Support:** Frameworks like Django, Flask, and FastAPI simplify enterprise application development.
- **Database Integration:** Easily integrates with databases such as PostgreSQL, MySQL, and Oracle.
- **Security Features:** Frameworks provide built-in security mechanisms.
- **API Development:** Excellent support for building RESTful APIs and microservices.
- **Community Support:** Large developer community ensures continuous improvements and support.

Enterprise Use Cases

- Web Applications
- ERP Systems
- CRM Systems
- Data Analytics Platforms
- Cloud Applications
- Automation Solutions
- API Services

Introduction to Django Framework

Django is a high-level, open-source Python web framework that enables developers to build secure, scalable, and maintainable web applications quickly.

It follows the **MVT (Model-View-Template)** architectural pattern and provides built-in features for database management, authentication, URL routing, form handling, and administration.

Key Features

- Rapid Development
- MVT Architecture
- Built-in Admin Panel
- ORM (Object-Relational Mapping)
- Authentication and Authorization
- URL Routing
- Security Features (CSRF, XSS, SQL Injection Protection)
- Scalability and Reusability

Components of Django

Model

- Handles database structure and data operations. **View**
- Contains business logic and processes requests.

Template

- Handles the presentation layer and user interface.

Advantages of Django

- Faster development process
- Clean and maintainable code
- Strong security features
- Easy database integration
- Large community support
- Suitable for enterprise-level applications

Common Use Cases

- E-commerce Applications
- Content Management Systems (CMS)
- Social Media Platforms
- Enterprise Applications
- REST APIs
- Data-Driven Web Applications

Introduction to PostgreSQL Database:

PostgreSQL is a powerful, open-source, object-relational database management system (ORDBMS) used to store, manage, and retrieve structured data efficiently.

It is known for its reliability, scalability, security, and support for advanced SQL features, making it a popular choice for enterprise applications.

Key Features

- Open Source and Free
- ACID Compliance (ensures data integrity)
- Advanced SQL Support
- High Performance
- Scalability
- Strong Security Features
- Support for Complex Queries
- Cross-Platform Compatibility

Components of PostgreSQL

Database

- A collection of related data.

Row (Record)

- Represents a single data entry.

Column (Field)

- Represents a specific attribute of data.

Schema

- Organizes database objects such as tables and views.

Advantages of PostgreSQL

- Reliable and stable
- Supports large-scale applications
- Excellent integration with Django
- Supports transactions and concurrency
- Advanced indexing and query optimization

Common Use Cases

- Web Applications
- Enterprise Systems
- E-commerce Platforms
- Financial Applications
- Data Warehousing
- Analytics Solutions

Software Development Life Cycle (SDLC):

Software Development Life Cycle (SDLC) is a structured process used to plan, design, develop, test, deploy, and maintain software applications efficiently and systematically.

It helps teams deliver high-quality software within the required time and budget.

Phases of SDLC

1. Requirement Gathering and Analysis • Collect business and user requirements.

- Analyze project feasibility and scope.

2. System Design

- Create the software architecture and design.
- Define database structure, user interfaces, and system components.

3. Development

- Write the application code based on the design specifications.
- Implement features and functionalities.

4. Testing

- Verify that the software works as expected.
- Identify and fix bugs and defects.

5. Deployment

- Release the application to the production environment.
- Make it available to end users.

6. Maintenance

- Monitor application performance.
- Fix issues, apply updates, and add new features.

Benefits of SDLC

- Improves software quality
- Reduces development risks
- Enhances project management
- Ensures timely delivery
- Facilitates better communication among stakeholders

Common SDLC Models

- Waterfall Model
- Agile Model
- Spiral Model
- V-Model

- Iterative Model

Agile & Jira workflow in companies

Agile Methodology

Agile is a software development methodology that focuses on iterative development, collaboration, continuous feedback, and delivering working software in small increments called **Sprints**.

Key Principles of Agile

- Customer collaboration
- Continuous improvement
- Frequent software delivery
- Adaptability to changing requirements
- Team collaboration and transparency

Agile Roles

Product Owner

- Defines requirements and prioritizes the backlog.

Scrum Master

- Facilitates Agile processes and removes obstacles.

Development Team

- Designs, develops, and tests the application.

Sprint

A **Sprint** is a fixed time period (usually 1–4 weeks) during which the team completes a set of planned tasks.

Sprint Workflow

1. Sprint Planning
2. Development
3. Daily Stand-up Meetings
4. Testing
5. Sprint Review
6. Sprint Retrospective

Jira Workflow in Companies

Jira is a project management and issue-tracking tool widely used in Agile teams to manage tasks, bugs, and project progress.

Typical Jira Workflow
Backlog
↓
To Do
↓
In Progress
↓
Code Review
↓
Testing / QA
↓
Done

Workflow Stages

Backlog

- List of all project requirements and tasks.

To Do

- Tasks selected for the current sprint.

In Progress

- Developer is actively working on the task.

Code Review

- Code is reviewed by senior developers or team members.

Testing / QA

- Quality Assurance team verifies the functionality.

Done

- Task is completed, tested, and approved.

Daily Agile Activities

Sprint Planning

- Team selects tasks from the backlog.

Daily Stand-up

- Team members discuss:

- What they completed yesterday.
- What they will work on today.

- Any blockers or issues.

Sprint Review

- Demonstrate completed work to stakeholders.

Sprint Retrospective

- Discuss improvements for future sprints.

Module 2: Development Environment Setup:

Task 1: Install and Configure Development Tools

Install and configure: •

Python 3.x

Download Python from official site: https://www.python.org/

During installation:

Check “Add Python to PATH”

Verify installation:

python --version

- ***VS Code***

  Download VS Code:
  https://code.visualstudio.com/

- **Install extensions:** Python (Microsoft)

  Pylance GitHub

  Extension

- **Verify:**

  Open VS Code
  Create .py file
  Run simple python program

- ***PostgreSQL***

  Download PostgreSQL:
  https://www.postgresql.org/download/

- **During installation**

  Set **username: postgres**

  Set **password (remember it)**

  Keep default port: **5432**

- ***Git & GitHub***

  Install Git

  Download Git: https://git-scm.com/

  Verify: git

  –version

- **Configure GitHub**

  Set username:

  git config --global user.name "Your Name"

  Set email:

  git config --global user.email your-email@example.com

  Verify:

  git config --list

- **Postman**

  Download Postman: https://www.postman.com/downloads/

  Verify:

  Open Postman

  Send sample request:

  GET https://api.github.com

- PgAdmin

  Download PostgreSQL:
  https://www.postgresql.org/download/

Task 2: Create Professional Project Structure

Create the following workspace:

Backend-Training/

│

├── Python-Practice/

│

├── Django-Projects/

│

├── PostgreSQL-Practice/

│

├── Notes/

│

└── README.md

Module 3: Python Fundamentals

Data Types:

Python provides various built-in data types such as Integer, Float, String, Boolean, List, Tuple, Dictionary, and Set. These data types are used to store and manage different kinds of data efficiently within an application.

Data Type	**Description**	**Example**
Integer (int)	Stores whole numbers without decimals.	10, -5, 100
Float (float)	Stores numbers with decimal points.	10.5, 3.14
String (str)	Stores a sequence of characters enclosed in quotes.	"Python"
Boolean (bool)	Stores logical values.	True, False
List	Ordered, mutable collection of items.	[1, 2, 3]
Tuple	Ordered, immutable collection of items.	(1, 2, 3)
Dictionary (dict)	Stores data as key-value pairs.	{"name": "John"}

Set	Unordered collection of unique items.	{1, 2, 3}

Operators in Python:

Operators in Python are used to perform operations on variables and values. Common categories include Arithmetic Operators for calculations, Comparison Operators for value comparison, Logical Operators for combining conditions, and Assignment Operators for assigning values to variables.

Operator Type	**Description**	**Examples**
Arithmetic Operators	Perform mathematical calculations.	+, -, *, /, %
Comparison Operators	Compare two values and return True or False.	==, !=, >, <, >=, <=
Logical Operators	Combine multiple conditions.	and, or, not
Assignment Operators	Assign values to variables.	=, +=, -=, *=, /=

Control Statements:

Control statements in Python manage the program flow. The if else statement is used for decision-making, the for loop is used for iterating over a sequence, and the while loop is used to execute a block of code repeatedly while a condition remains true.

Statement	**Description**
if else	Executes different blocks of code based on a condition.
for loop	Repeats a block of code for each item in a sequence.
while loop	Repeats a block of code as long as a condition is true.

if else

Used for decision-making based on conditions.

Example:

```
  if age >= 18:
print("Eligible")
else:
```

```
  print("Not Eligible") for
```

loop

Used when the number of iterations is known.

```
  for i in range(5):
print(i)
```

while loop
Used when the loop should continue until a condition becomes false.

```
  count = 1    while
count <= 5:
  print(count)
count += 1
```

Functions

A **Function** is a reusable block of code designed to perform a specific task. Functions help improve code reusability, readability, and maintainability.

Topic	**Description**
Creating Functions	Define a function using the def keyword.
Parameters	Inputs passed to a function to perform operations.
Return Values	Output returned from a function using the return statement.

Creating Functions

Used to define a reusable block of code.

```
  def greet():
  print("Hello, Welcome!")
```

Parameters

Parameters allow functions to accept input values.

```
  def greet(name):
print("Hello", name)
```

Return Values

Return values send the result back to the function caller.

```
  def add(a, b):
return a + b
```

Practical Coding Tasks

Task 3: Employee Management Console Application

Create a Python program for storing employee details:

Employee fields:

Employee Fields

- Employee ID
- Name
- Email
- Department
- Salary
- Experience **Features**

Add Employee

- Add new employee details to the system.

View All Employees

- Display all employee records.

Search Employee by ID

- Retrieve employee details using Employee ID.

Update Employee Details

- Modify existing employee information.

Delete Employee

- Remove an employee record from the system.

Exit Application

- Terminate the program safely.

Concepts to use:

Functions

- Separate functionality into reusable functions such as Add, Search, Update, and Delete.

List of Dictionaries

- Store multiple employee records in a list.

- Each employee record is represented as a dictionary.

Conditions

- Validate user choices and employee existence.

Loops

- Continuously display the menu until the user chooses to exit.

Exception Handling

- Handle invalid inputs and runtime errors gracefully.

Data Structure Example

```
employees = [
{
"id": 101,
"name": "John",
"email": "john@gmail.com",
"department": "Backend",
"salary": 50000,
"experience": 3
}
]
```

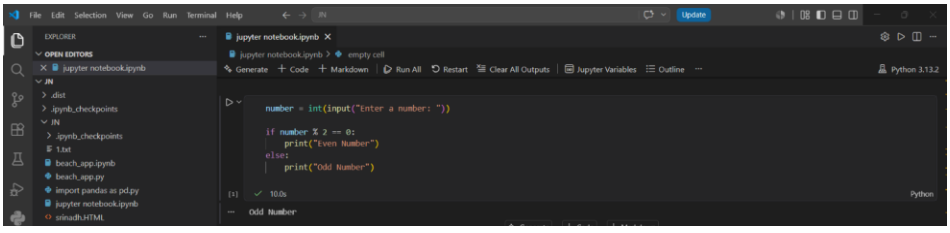
Explanation

This Employee Management System is a console-based Python application that uses Functions, List of Dictionaries, Conditions, Loops, and Exception Handling to perform CRUD operations such as Add, View, Search, Update, and Delete employee records. It demonstrates fundamental Python programming concepts and data management techniques.

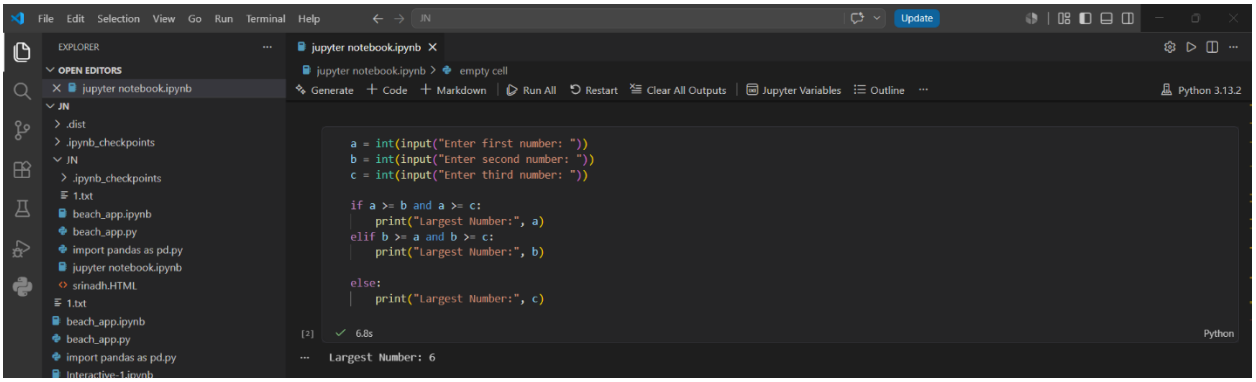
Task 4: Python Coding Challenges

Easy

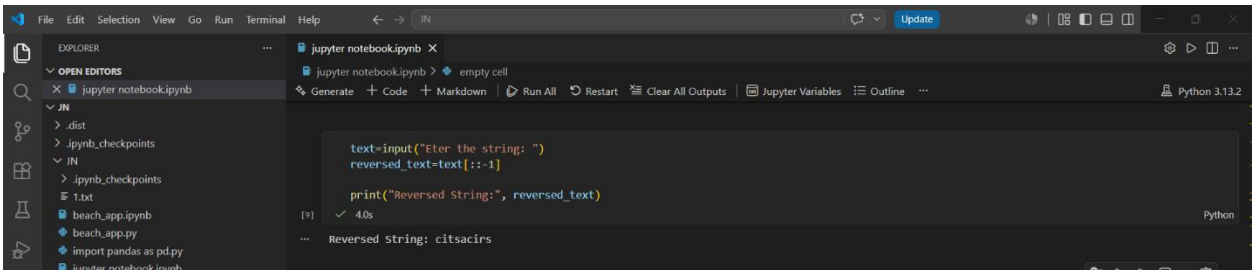
1.Check whether a number is even or odd.



2.Find the largest of three numbers.



3.Reverse a string.

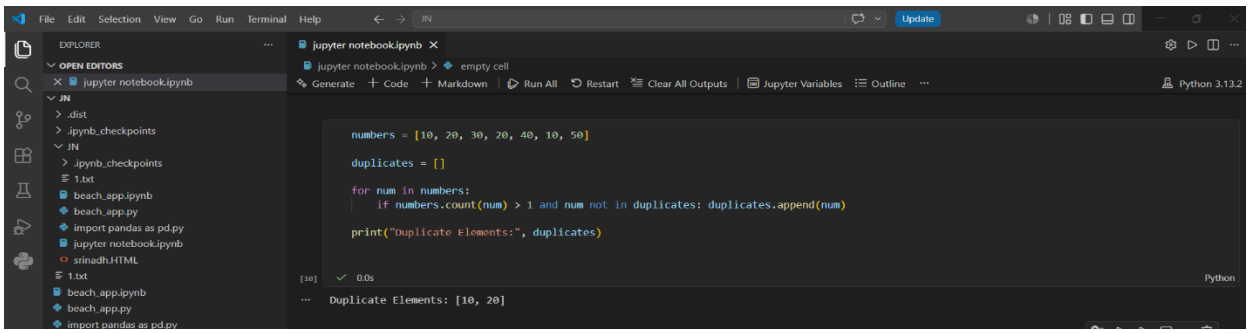


4.Count vowels in a string.

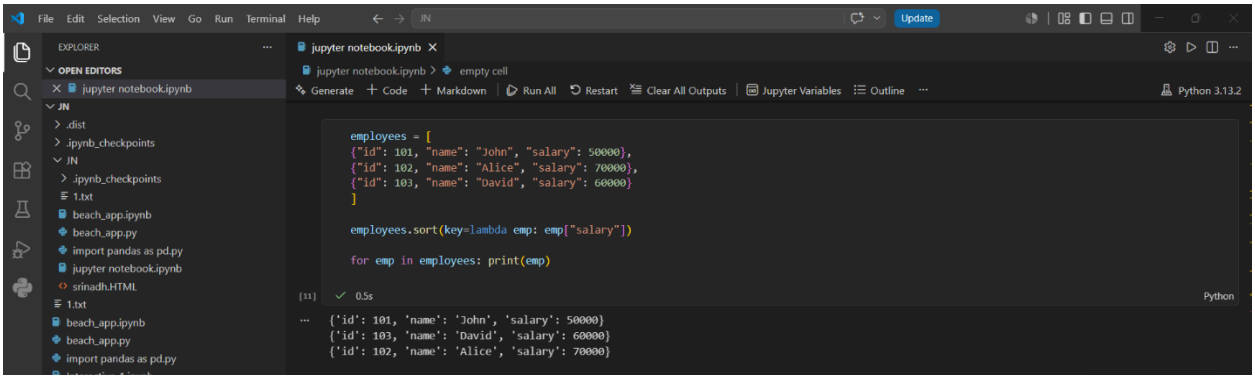
5.Check a palindrome.

Medium

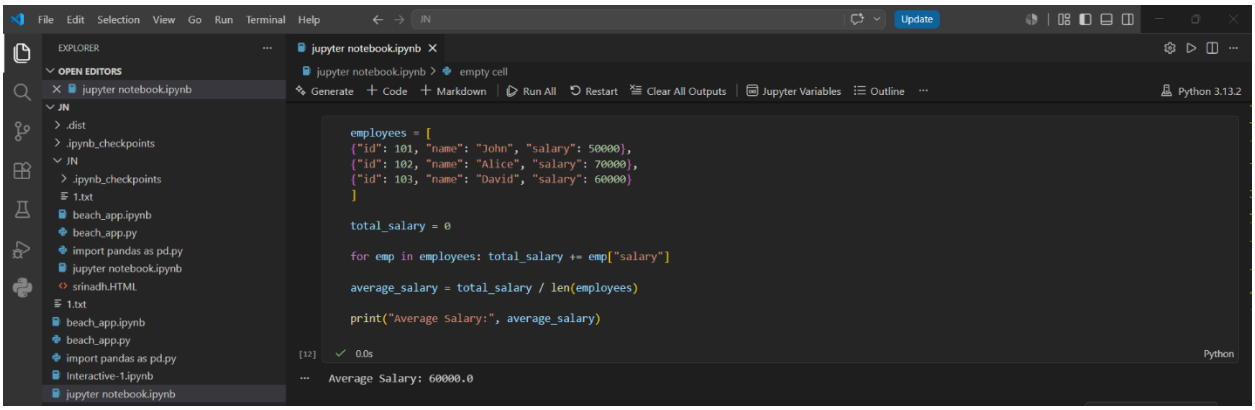
1.Find duplicate elements in a list.



2.Sort employee salaries.



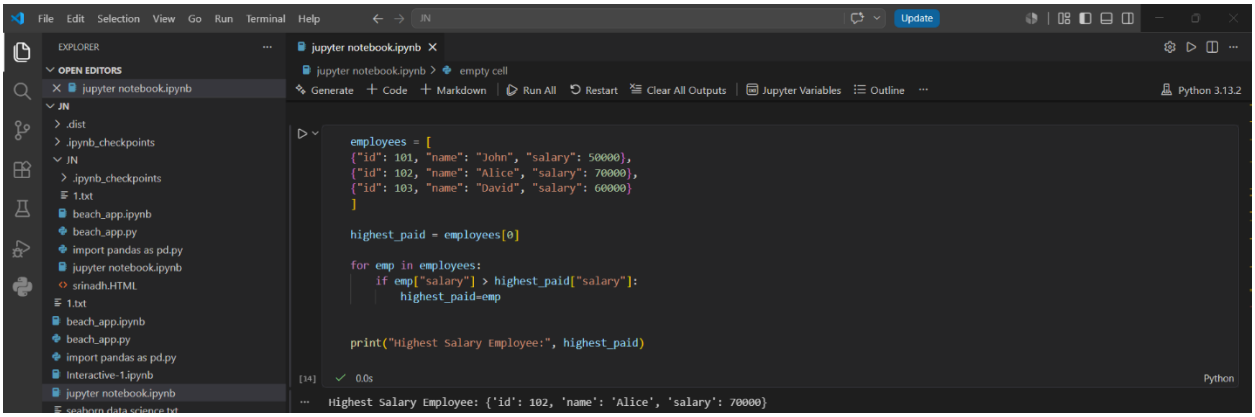
3. Calculate average salary from employee records.



```
employees = [
{"id": 101, "name": "John", "salary": 50000},
{"id": 102, "name": "Alice", "salary": 70000},
{"id": 103, "name": "David", "salary": 60000}
]

total_salary = 0

for emp in employees: total_salary += emp["salary"]

average_salary = total_salary / len(employees)

print("Average Salary:", average_salary)
```

```
Average Salary: 60000.0
```

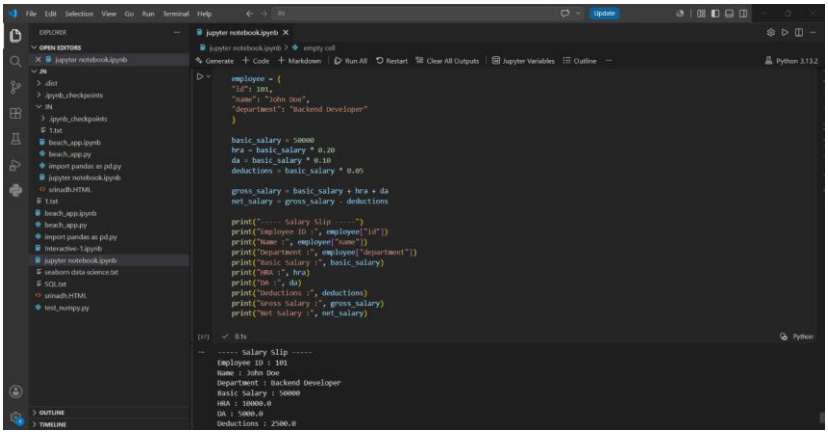
4.Find the employee with highest salary.



```
employees = [
{"id": 101, "name": "John", "salary": 50000},
{"id": 102, "name": "Alice", "salary": 70000},
{"id": 103, "name": "David", "salary": 60000}
]

highest_paid = employees[0]

for emp in employees:
    if emp["salary"] > highest_paid["salary"]:
        highest_paid=emp


print("Highest Salary Employee:", highest_paid)
```

```
Highest Salary Employee: {'id': 102, 'name': 'Alice', 'salary': 70000}
```

5.Generate a simple salary slip.



```
employee = {
"id": 101,
"name": "John Doe",
"department": "Backend Developer"
}

basic_salary = 50000
hra = basic_salary * 0.20
da = basic_salary * 0.10
deductions = basic_salary * 0.05

gross_salary = basic_salary + hra + da
net_salary = gross_salary - deductions

print("----- Salary Slip -----")
print("Employee ID :", employee["id"])
print("Name :", employee["name"])
print("Department :", employee["department"])
print("Basic Salary :", basic_salary)
print("HRA :", hra)
print("DA :", da)
print("Deductions :", deductions)
print("Gross Salary :", gross_salary)
print("Net Salary :", net_salary)
```

```
----- Salary Slip -----
Employee ID : 101
Name : John Doe
Department : Backend Developer
Basic Salary : 50000
HRA : 10000.0
DA : 5000.0
Deductions : 2500.0
```

Module 4: Introduction to Git Workflow

Practical Task

Create a GitHub repository:

Go to GitHub and create a repository:

Repository Name:

python-backend-training

- Set it to **Public or Private**
- Do NOT initialize with README

Open Terminal / Command Prompt

- Go to your project folder:
- cd path/to/your/project

Initialize Git

- git init

Add Files to Git

- git add .

Commit Code (Initial Commit)

- git commit -m "Initial commit - added backend training files"

Connect to GitHub Repository

- git remote add origin https://github.com/srindhatla/python-backend-training.git

Push Code to Main Branch

- git branch -M main
- git push -u origin main

Create Branches
Create Development Branch

- git checkout -b development
- git push -u origin development

Create Feature Branch

- git checkout -b feature/employee-management git push -u origin feature/employee-management

Git Workflow Summary

- main
  ├── development
  └── feature/employee-management

Submission Requirements (Company Standard)

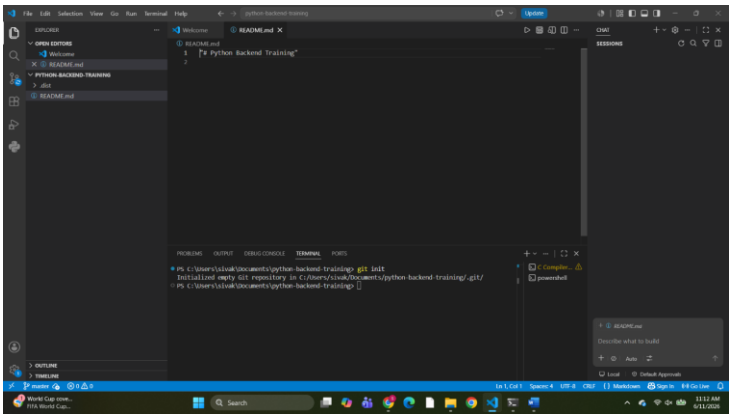
GitHub Repository Link Submit

your repository link:

https://github.com/srinadhatla/python-backend-

GitHub repository

Project structure screenshot

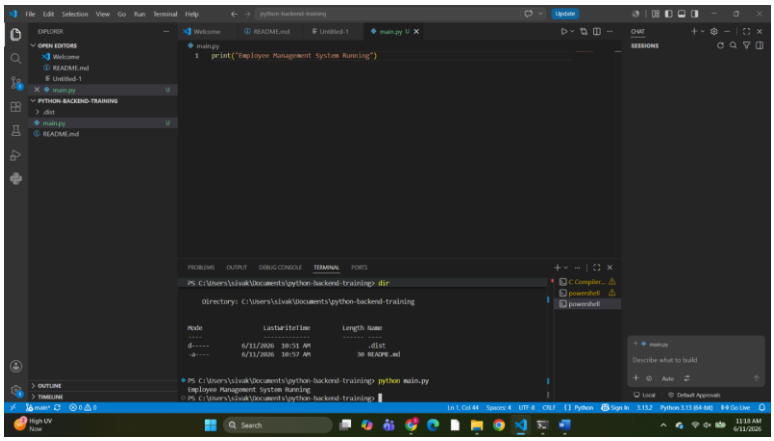


project should follow this structure

python-backend-training/
│
├── employee_management/

```
│   ├── main.py
│   ├── employees.py
│   ├── utils.py
│
├── README.md
│
└── requirements.txt
```

Python Employee Management Output Screenshot



Project Overview

This is a console-based Employee Management System built using Python. It allows users to perform CRUD operations such as adding, viewing, searching, updating, and deleting employee records using in-memory data structures.

Features

- Add Employee
- View All Employees
- Search Employee by ID
- Update Employee Details
- Delete Employee
- Exit Application

Technologies Used

- Python 3
- Functions
- List of Dictionaries
- Conditional Statements
- Loops
- Exception Handling

How to Run the Project

Step 1: Clone Repository

git clone

Step 2: Open terminal in project folder

cd python-backend-training

Step 3: Run Python File

python main.py

Evaluation Criteria

Skill	**Marks**
Environment Setup	10
Python Basics	20
Problem Solving	20
Employee Management Application	30
Git Workflow	10
Documentation	10
Total	100

Day 1 Deliverable (IRA/Jira Style)

Story: Backend Developer Training — Day 1 Setup & Python Fundamentals

Description:

As a backend trainee, I need to set up the complete development environment and build Python console applications so that I can understand the foundation required for enterprise Django development.

Acceptance Criteria:

- Python, PostgreSQL, VS Code, Git, and Postman are installed.
- Professional project folder structure is created.
- Python fundamentals programs are completed.
- Employee Management System supports complete CRUD operations.
- Code is pushed to GitHub using proper Git workflow.
- README documentation is completed.

Priority:

High

Estimated Time:

8 Hours